Efficient and Privacy-Preserving Federated Learning Against Poisoning Adversaries

Jiaqi Zhao, Hui Zhu, *Senior Member, IEEE*, Fengwei Wang, Yandong Zheng, Rongxing Lu, *Fellow, IEEE*, and Hui Li, *Member, IEEE*

***Abstract*—The ever-growing data scale and increasingly strict privacy restraint have recently drawn extensive attention to federated learning (FL) as a multi-party machine learning paradigm for achieving high-quality model construction without data collection. Nevertheless, uploading local models in FL can still be exploited by adversaries to infer participants' sensitive data. Furthermore, it is possible for malicious participants to manipulate the global model by submitting poisonous local models. To tackle these challenges, this article proposes an efficient and privacy-preserving federated learning framework against poisoning adversaries, namely ELFL, which can ensure the confidentiality of local models while effectively resisting data poisoning attacks. Specifically, we first design a grouped secure aggregation algorithm, through which the aggregation server can compute the summations of local models inside logic groups but cannot see individual ones. Then, based on grouped aggregations, our poisoning defense mechanism could detect and quickly phase out malicious participants from training candidates. Moreover, the computational complexity of participants is independent of their total number, so it is suitable for large-scale scenes. Detailed security analysis demonstrates the security of ELFL. Experimental results show that ELFL could maintain a high accuracy against representative data poisoning attacks, and its computational and communication overhead is indeed low.**

***Index Terms*—Federated learning, privacy preservation, poisoning resistance, efficiency.**

I. Introduction

In RECENT years, federated learning (FL) [1], [2] has gained considerable attention as a promising multi-party machine learning paradigm that allows multiple participants to construct a high-quality model collaboratively. In the FL framework, an aggregation server holds the global model, which is trained locally by participants and updated periodically by aggregating these local models. Benefiting from the advantage that training data are controlled locally by participants, FL takes a big leap forward in mitigating privacy risks compared to centralized machine learning.

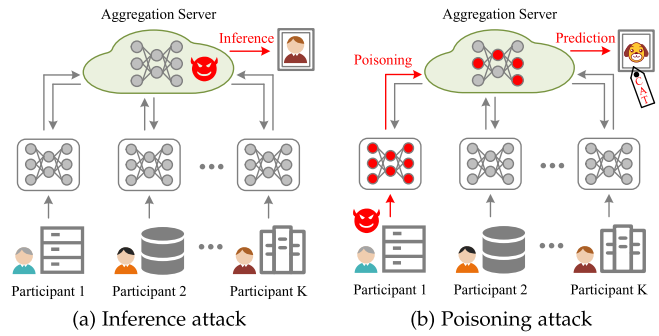

Fig. 1. Privacy and security issues in FL.

Despite its promising prospects, FL still faces many privacy and security challenges [2], [3]. As illustrated in Fig. 1, since the local models in FL reveal the data characteristic of participants, an adversarial aggregation server can infer or even recover the training samples from them [4], [5]. Moreover, participants' behavior in FL is uncontrolled due to the distributed architecture, so malicious participants may attempt to manipulate the global model by submitting poisonous local models trained by crafted data, i.e., data poisoning attacks [6], [7]. With the proposal of many FL-adapted secure aggregation (SA) algorithms [8], [9], the former issue can be resolved by aggregating local models while ensuring their individual invisibility. However, the poisoning defense methods [10], [11] usually require analysis of the similarity between individual local models in plaintexts, so there is a contradiction in simultaneously considering both privacy and security in FL.

To address the aforementioned contradiction, a number of privacy-preserving and poisoning-resistant FL schemes have been proposed recently. Schemes [12], [13], [14] use homomorphic encryption (HE) as the underlying privacy-preserving technique, and they design cipher-based protocols to detect poisonous models, which are executed over two non-collusive servers. In schemes [15], [16], [17], the local models are secretly shared with multiple servers based on the secure multi-party computation (MPC) technique, and it utilizes boolean or arithmetic circuits to securely detect outlier models. On the one hand, expensive computational or communication overhead is needed for analyzing massive models over

Manuscript received 20 July 2023; revised 11 March 2024; accepted 12 March 2024. Date of publication 20 March 2024; date of current version 9 October 2024. This work was supported in part by the National Natural Science Foundation of China under Grant U22B2030 and Grant 62302360, in part by Ant Group, in part by Shaanxi Provincial Key Research and Development Program under Grant 2023-ZDLGY-35, in part by 111 Project under Grant B16037, in part by the Natural Science Basic Research Plan in Shaanxi Province of China under Grant 2023-JC-QN-0699, and in part by the Fundamental Research Funds for the Central Universities under Grant ZYTS23164. *(Corresponding author: Hui Zhu.)*

Jiaqi Zhao, Hui Zhu, Fengwei Wang, Yandong Zheng, and Hui Li are with the School of Cyber Engineering, Xidian University, Xi'an, Shaanxi 710126, China (e-mail: jq_zhao@stu.xidian.edu.cn; zhuhui@xidian.edu.cn; wangfengwei@xidian.edu.cn; zhengyandong@xidian.edu.cn; lihui@mail.xidian.edu.cn).

Rongxing Lu is with the Faculty of Computer Science, University of New Brunswick, Fredericton, NB E3B 5A3, Canada (e-mail: rlu1@unb.ca).

Digital Object Identifier 10.1109/TSC.2024.3377931

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

ciphertexts or secret slices. On the other hand, the setting of multiple non-collusive servers may not be practical in reality. We note that the closest to ours are schemes SAFELearning [18] and SBRDL [19], and they divide participants into groups to execute two-step or multi-step aggregation, which can utilize intermediate aggregation results for poisoning detection. However, both of them adopt the double masking technique to securely aggregate in-group local models for privacy preservation, and its computational complexity is also as high as $\mathcal{O}(L^2)$, where L is the number of participants in one group. Moreover, they do not support dynamic grouping well, which leads to an additional re-clustering overhead in SBRDL, and causes SAFELearning to be unable to detect poisoning attacks in earlier training rounds due to changeless groups.

Therefore, in this paper, we propose an efficient and privacy-preserving federated learning framework against poisoning adversaries, namely ELFL, where participants are divided into logic groups for model training, and we specially design a grouped secure aggregation (g-SA) algorithm and a poisoning defense mechanism to efficiently ensure the privacy and security in in-group and inter-group aggregation, respectively. Specifically, the contributions of ELFL are threefold.

• *ELFL guarantees the privacy of local models against inference attacks:* We propose a new grouped secure aggregation algorithm for local models, by which one aggregation server can only obtain the summation of in-group models exceeding a certain threshold but not individual models, thus preventing inference attacks. Moreover, the g-SA algorithm supports participants quitting or joining at any training round.

• *ELFL ensures the security of the global model against malicious participants:* We design a suite of poisoning defense methods on the basis of grouped aggregations. Specifically, our poisoning detection method could effectively find out aggregations containing malicious local models. Then, based on the detection results, our random grouping method and reputation system could quickly phase out malicious participants joining training to obtain a high-quality global model.

• *ELFL is efficient in computational cost and communication overhead:* First, ELFL detects poisoning attacks at the level of aggregations, which does not need complex cryptographic operations and multi-cloud collaboration, achieving a trade-off between security and efficiency. Second, ELFL divides participants into logical groups rather than actual ones, where center participants are not required, and one aggregation server can directly obtain the aggregations of all groups, so there is no communication required between participants in ELFL. Third, ELFL supports dynamic grouping, and all participants continue to use the same suite of keys even though the participant or grouping changes, eliminating the overhead of key re-initialization for each random grouping. Moreover, the computational complexity of each participant in ELFL is independent of the number of participants, which will significantly decrease their computational cost. Extensive experiments conducted on real-world datasets demonstrate that ELFL is indeed efficient, especially for the scenes of large-scale participants.

The rest of this paper is organized as follows. In Section II, we define the system and security models. Section III introduces some building blocks of our framework. Section IV describes our framework from a high-level perspective. Section V presents the technical details of our framework, followed by the security and complexity analysis in Section VI and performance evaluation in Section VII. Finally, we review some related works in Section VIII and draw a conclusion in Section IX.

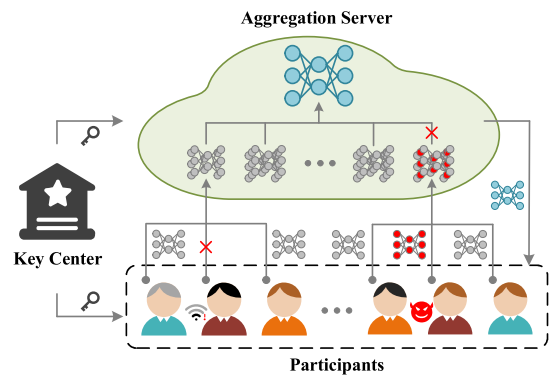

Fig. 2. System model under consideration.

II. System and Security Models

This section defines the system and security models.

A. System Model

Our framework concentrates on collaboratively constructing a global deep learning model over multiple participants, which consists of three entities, including the key center (KC), the aggregation server (AS), and multiple participants ($\mathcal{L}$), as shown in Fig. 2.

Key Center: KC is in charge of key management (containing the key application, assignment, distribution, storage, and revocation) and participant management (containing active participant management and participant reputation management).

Aggregation Server: AS is an aggregation server equipped with powerful computing resources, responsible for grouped local model aggregating, poisoning detection, and global model updating. In order to improve the accuracy of poisoning detection, we assume that AS can manipulate a small part of public validation samples, written as $\mathcal{D}_v$, which are independent and identically distributed with the training data.

Participants: In ELFL, each participant $\mathrm{P}_{id} \in \mathcal{L}$ has a part of training samples $\mathcal{D}_{id} \subset \mathcal{D}$, and it trains and sends the local model to AS periodically. In particular, during each round, participants in ELFL are divided into different logic groups for training. Moreover, there may be some new participants $\mathrm{P}_{id_{new}}$ joining training or some participants $\mathrm{P}_{id_{old}}$ dropping training due to device or network failures.

B. Security Model

In our security model, KC is fully trusted. AS is considered honest-but-curious [20], i.e., it will honestly follow the stipulated protocol like model detection and aggregation but try to infer participants' data as much as possible. For instance, AS may attempt to recover raw training samples based on received local models [4]. What's more, AS may collude with a few corrupted participants to improve its inference ability.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

According to participants' behavior, they are classified into malicious and benign. Without loss of generality, we assume that the number of malicious participants is T. Benign participants honestly submit their local models, which are obtained by executing the model training algorithm on their true training samples, but malicious participants individually or collaboratively launch the data poisoning attack by training local models on poisonous data [7], [21], [22]. Moreover, there are some other security threats in FL, e.g., denial of service attacks and free-riding attacks, but they are currently beyond the scope of this work and may be considered in our future work. Under the defined security model, our framework should satisfy the following security requirements.

Privacy Preservation: Participants' data samples contain massive amounts of sensitive information, the privacy of which is the first-order concern in FL. Hence, ELFL should strictly protect the sensitive data of participants against various inference attacks.

Poisoning Resistance: Data poisoning attacks could significantly degrade the model performance or even lead the model convergence to a direction set by the adversary. ELFL should provide the ability to detect poisonous models and weaken their damage to the global model.

III. Preliminaries

This section briefly introduces definitions of federated learning, Paillier encryption, and threshold secret sharing.

A. Federated Learning

Federated learning [1], [2] is a distributed machine learning paradigm, which involves utilizing multi-party data to construct high-quality models while keeping them localized. Federated averaging [1] is a classic FL algorithm executed iteratively between one aggregation server and several participants, and each training round runs in two steps.

- *Local training:* During the r-th round, AS randomly selects a partition of participants $\mathcal{L}^{(r)}$ and sends the global model $W^{(r)}$ to them. Every participant $\mathrm{P}_{id} \in \mathcal{L}^{(r)}$ parallelly executes E-epoch stochastic gradient descent (SGD) algorithm [23] to minimize the loss $l(W_{id}^{(r)}; \mathcal{D}_{id})$ and obtains the local model $W_{id}^{(r)}$, which is uploaded to AS.
- *Model updating:* AS aggregates local models from $\mathcal{L}^{(r)}$ as the new global model $W^{(r+1)} = \sum_{\mathrm{P}_{id} \in \mathcal{L}^{(r)}} \frac{|\mathcal{D}_{id}|}{|\mathcal{D}|} W_{id}^{(r)}$.

B. Paillier Encryption

Paillier encryption [24] is a public-key cryptosystem based on the composite residuosity problem, which supports homomorphic additions and scalar multiplications. Paillier encryption contains the following functi ons.

- *Key generation:* Given the security parameter κ, select two large primes p and q with $\frac{\kappa}{2}$ bits, compute $\mathcal{N} = p \cdot q$ and $\lambda = \mathrm{lcm}(p-1, q-1)$. Then, select a random integer $g \in \mathbb{Z}^*_{\mathcal{N}^2}$ satisfying $\gcd(L(g^\lambda \bmod \mathcal{N}^2), N) = 1$, where $L(u) = \frac{u-1}{\mathcal{N}}$ for $u \in \mathbb{Z}^*_{\mathcal{N}^2}$. Finally, output the public key $pk = (\mathcal{N}, g)$ and secret key $sk = (\lambda)$.
- *Encryption:* Given a message m, the random number $r \in \mathbb{Z}^*_{\mathcal{N}^2}$ is chosen and the ciphertext can be computed as
$$[\![m]\!]_{pk} = \mathrm{Enc}(m, pk) = g^m r^{\mathcal{N}} \bmod \mathcal{N}^2.$$
- *Decryption:* Given the ciphertext $[\![m]\!]_{pk} < \mathcal{N}^2$, the message m can be retrieved as
$$m = \mathrm{Dec}([\![m]\!]_{pk}, sk) = \frac{L((\![m]\!]_{pk})^\lambda \bmod \mathcal{N}^2)}{g^\lambda \bmod \mathcal{N}^2} \bmod \mathcal{N}.$$
- *Homomorphic properties:* Given two ciphertexts $[\![m_1]\!]_{pk}, [\![m_2]\!]_{pk}$ and one plaintext m_3, the homomorphic properties of the Paillier encryption can be represented as $\mathrm{Dec}([\![m_1]\!]_{pk} \cdot [\![m_2]\!]_{pk}, sk) = m_1 + m_2$ and $\mathrm{Dec}(([\![m_1]\!]_{pk})^{m_3}, sk) = m_1 \cdot m_3$.

C. Threshold Secret Sharing

Threshold secret sharing [25] is a cryptographic algorithm used for distributing a secret c into K shares, and the secret will not be revealed unless shares exceeding the threshold S are combined, which contains two functions.

- *Sharing:* Select a big prime number q, and select $S-1$ random coefficients $\{a_1, \ldots, a_{S-1}\}$ that are less than q. Construct an $(S-1)$-degree polynomial
$$f(x) = c + a_1 \cdot x + a_2 \cdot x^2 + \cdots + a_{S-1} \cdot x^{S-1} \bmod q.$$
Then, evaluate $f(x)$ on K points $\{x_1, \ldots, x_K\}$, and $\{f(x_1), \ldots, f(x_K)\}$ are secret shares.
- *Reconstruction:* If H secret shares $\{x_1, \ldots, x_H\} \subset \{x_1, \ldots, x_K\}$ are obtained ($H \geq S$), it can reconstruct
$$f(x) = \sum_{h=1}^{H} f(x_h) \prod_{i=1,\ldots,k}^{i \neq h} \frac{x - x_i}{x_h - x_i},$$
and $f(0)$ is the secret c.

IV. Overview

This section introduces the high-level execution flow of ELFL, as shown in Fig. 3, which consists of three phases: system initialization, local training, and model updating. In particular, the g-SA algorithm can provide grouped aggregations in the model updating phase, which can be utilized for poisoning detection and defense. Moreover, the commonly used symbols are listed in Table I.

A. System Initialization

Our framework is initialized via participant registration and parameter generation.

• *Step 1: Participant registration*

In this step, participants submit registration applications to KC, intending to join the FL system. Once approved, the participant is given a unique and random identifier id, denoted as P_{id}. Moreover, KC maintains an active participant list $\mathcal{L}$ and a participant reputation list $\mathcal{R}$.

• *Step 2: Parameter initialization*

Given the security parameter κ, KC first generates and distributes keys of the g-SA algorithm, where the aggregation

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

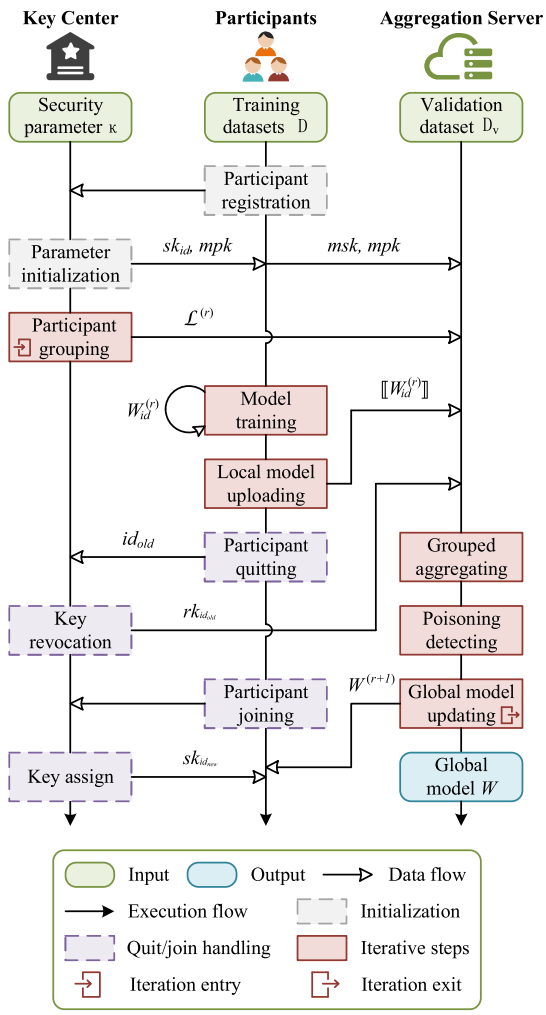

Fig. 3. Overview of ELFL.

TABLE I
DEFINITION OF NOTATIONS IN ELFL

Symbols	Description
P_{id}	The participant with the identifier id.
$\mathcal{L}$	The active participant list.
$\mathcal{R}$	The participant reputation list.
$\mathcal{L}^{(r)}$	The selected participant list in the r-th training round.
$\mathcal{M}^{(r)}$	The list of detected malicious groups.
$\mathcal{G}_n^{(r)}$	The participant list of the n-th group.
T	The number of malicious participants.
N	The number of groups.
L	The number of participants in each group.
K	The number of active participants in $\mathcal{L}$.
S	The minimum number of participants in each group.
M	The model size.
ρ	The ratio of selected participants in each round.
msk	The aggregation key.
sk_{id}	The secret key of P_{id}.
$W^{(r)}$	The global model in the r-th training round.
$W_{id}^{(r)}$	The local model of P_{id} in the r-th training round.
$A_n^{(r)}$	The grouped aggregation in $\mathcal{G}_n^{(r)}$.
κ, ϵ	The security and precision parameters.
α, E	The learning rate and local epoch.
$[\![W_{id}^{(r)}]\!]$	The ciphertext local model of P_{id}.

key msk is sent to AS, and the secret keys $\{sk_{id}\}_{\mathrm{P}_{id}\in\mathcal{L}}$ are distributed to corresponding participants. Then, AS randomly initializes the global model $W^{(1)}$, and selects the learning rate α, precision parameter ϵ, participant selection ratio ρ, and local training epoch E.

B. Local Training

After system initialization, multiple rounds of local training and model updating are performed iteratively until the global model reaches convergence.

• *Step 1: Participant grouping*

At the r-th training round, according to the reputation list $\mathcal{R}$, KC selects a part of participants $\mathcal{L}^{(r)}$ from $\mathcal{L}$ with the ratio ρ. Then, based on our random grouping method, KC divides participants in $\mathcal{L}^{(r)}$ into multiple logic groups, written as $\mathcal{L}^{(r)} = \{\mathcal{G}_1^{(r)}, \ldots, \mathcal{G}_N^{(r)}\}$, which is sent to AS.

• *Step 2: Model training*

Each participant $\mathrm{P}_{id} \in \mathcal{L}^{(r)}$ locally executes E-epoch SGD training with $\mathcal{D}_{id}$ and obtains the local model $W_{id}^{(r)}$.

• *Step 3: Local model uploading*

Based on the g-SA algorithm, each participant $\mathrm{P}_{id} \in \mathcal{L}^{(r)}$ encrypts its $W_{id}^{(r)}$ with secret key sk_{id} and uploads the ciphertext local model $[\![W_{id}^{(r)}]\!]$ to AS.

C. Model Updating

This phase updates the global model after detecting and eliminating the poisonous local models.

• *Step 1: Grouped aggregating*

After receiving local models, AS computes the aggregations in different groups with the g-SA algorithm, denoted as $\{A_1^{(r)}, \ldots, A_N^{(r)}\}$.

• *Step 2: Poisoning detecting*

Based on our poisoning detection method, AS detects the poisonous grouped aggregations, which are recorded to a list $\mathcal{M}^{(r)}$, and the reputation list is also updated.

• *Step 3: Global model updating*

AS updates the global model as

$$W^{(r+1)} = \frac{\sum_{\mathcal{G}_n^{(r)} \notin \mathcal{M}^{(r)}, \mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} A_n^{(r)} \cdot |\mathcal{D}_{id}|}{\sum_{\mathcal{G}_n^{(r)} \notin \mathcal{M}^{(r)}, \mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} |\mathcal{D}_{id}|},$$

and sends $W^{(r+1)}$ to all active participants, where $|\mathcal{D}_{id}|$ means the number of samples in $\mathcal{D}_{id}$.

V. TECHNICAL DETAILS

In this section, we detailedly introduce the proposed grouped secure aggregation algorithm and poisoning defense mechanism.

A. Grouped Secure Aggregation (g-SA) Algorithm

Based on the Paillier encryption [24], the g-SA algorithm can achieve grouped secure aggregation over multiple participants, through which AS cannot see the messages of individual participants but can obtain aggregations of those in predefined groups.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Grouped Secure Aggregation.

Key Generation

Input: $\kappa, \mathcal{L}, S$ **Output:** $mpk, msk, \{sk_{id}\}_{\mathrm{P}_{id}\in\mathcal{L}}$

1: Given the security parameter κ, the key generation function first randomly selects two $\frac{\kappa}{2}$-bit primes p, q and computes $\mathcal{N} = p \cdot q$ and $\lambda = \mathrm{lcm}(p-1, q-1)$;
2: Select a base $g \in \mathbb{Z}^*_{\mathcal{N}^2}$ satisfying $\gcd(L(g^\lambda \bmod \mathcal{N}^2), \mathcal{N}) = 1$ and $L(x) = \frac{x-1}{\mathcal{N}}$;
3: Randomly select a $\frac{\kappa}{2}$-bit prime u and a κ-bit integer g', which satisfies $\gcd(u, \phi(\mathcal{N})) = 1$ and $v = g^u \bmod \mathcal{N}^2$;
4: Output $mpk = (g, g', v, \mathcal{N})$ and $msk = (\lambda, u)$;
5: Construct an $(S-1)$-degree polynomial $f(x) = a_1 \cdot x + a_2 \cdot x^2 + \cdots + a_{S-1} \cdot x^{S-1} \bmod q$, where coefficients $a_1, \cdots, a_{S-1}$ are random integers less than q;
6: For each $\mathrm{P}_{id} \in \mathcal{L}$, compute $sk_{id} = g'^{f(id)\cdot p} \bmod \mathcal{N}^2$. The identifier id of P_{id} is a unique random number less than q.

Message Encryption

Input: $W^{(r)}_{id}, \mathcal{L}^{(r)}, mpk, sk_{id}$ **Output:** $[\![W^{(r)}_{id}]\!]$

1: Given the group $\mathcal{G}^{(r)}_n \in \mathcal{L}^{(r)}$ that P_{id} belongs to, compute $\mu_{id} = -\prod^{id'\neq id}_{\mathrm{P}_{id'}\in\mathcal{G}^{(r)}_n} id' \cdot \prod^{id''<id'}_{\mathrm{P}_{id''}\in\mathcal{G}^{(r)}_n} (id' - id'') \bmod q$.
2: Compute the ciphertext as $[\![W^{(r)}_{id}]\!] = g^{W^{(r)}_{id}} v^\gamma sk_{id}{}^{\mu_{id}} \bmod \mathcal{N}^2$, where γ is a $(\frac{\kappa}{2}-1)$-bit random integer, which is re-selected for every encryption.

Grouped Aggregation

Input: $\{[\![W^{(r)}_{id}]\!]\}_{\mathrm{P}_{id}\in\mathcal{L}^{(r)}}, \mathcal{L}^{(r)}, mpk, msk$ **Output:** $\{A^{(r)}_n\}^N_{n=1}$

1: **for** $n = 1, \cdots, N$ **do**
2: Compute the ciphertext grouped aggregation $[\![A^{(r)}_n]\!] = \prod_{\mathrm{P}_{id}\in\mathcal{G}^{(r)}_n} [\![W^{(r)}_{id}]\!] \bmod \mathcal{N}^2$.
3: Output the grouped aggregation $A^{(r)}_n = (\frac{L([\![A^{(r)}_n]\!]^\lambda \bmod \mathcal{N}^2)}{L(g^\lambda \bmod \mathcal{N}^2)} \bmod \mathcal{N}) \bmod u$.

The g-SA algorithm consists of a tuple of functions, namely key generation, message encryption, and grouped aggregation, as shown in Algorithm 1.

Specifically, the individual messages in the g-SA algorithm are protected in the form of a variant of Paillier encryption, whose security is proved in Section VI-A. Second, since each participant's encryption parameter μ_{id} is an interpolation related to its group, messages from different groups cannot be aggregated. Third, the threshold secret sharing is used to split a secret 0 modulo q, and they are used to generate the secret keys of participants. Only aggregations with more than S ciphertexts in one group can recover the exponent $k \cdot N$ to trigger the trapdoor of Paillier encryption, and can be decrypted by msk. Moreover, during each r-th aggregation, each participant is only required to obtain an encryption parameter μ_{id}, and they cannot be used to infer the grouping rule $\mathcal{L}^{(r)}$, which reduces the advantage for adversaries launching collusive attacks.

Furthermore, no matter whether participants join in or drop out in a particular round, the g-SA algorithm does not need to re-execute the key generation and distribution, and they can be removed or added to the system via key revocation or assignment.

Participant Quitting: If one participant $\mathrm{P}_{id_{old}}$ drops out in a certain aggregation round due to equipment or network error, and the number of the remaining participants in its group is still more than S, a recovery key $rk_{id_{old}}$ will be assigned for it. KC first randomly selects a random number γ and computes $rk_{id_{old}} = v^\gamma (sk_{id_{old}})^{\mu_{id_{old}}}$. Then, messages of other participants in its group $\mathcal{G}^{(r)}_n$ can still be aggregated by computing $[\![A^{(r)}_n]\!] = \prod_{\mathrm{P}_{id}\in\mathcal{G}^{(r)}_n, id\neq id_{old}} [\![W^{(r)}_{id}]\!] \cdot rk_{id_{old}} \bmod \mathcal{N}^2$. After that, $\mathrm{P}_{id_{old}}$ is removed from the active participant list $\mathcal{L}$, and its secret key $sk_{id_{old}}$ is revoked. If it intends to rejoin the system, it will be considered a new participant and be assigned a new secret key.

Participant Joining: If one participant tries to join the system midway, it will be first assigned a unique identifier id_{new}. Then, $\mathrm{P}_{id_{new}}$ is added to the active participant list $\mathcal{L}$, and its secret key $sk_{id_{new}}$ is generated by computing $sk_{id_{new}} = g'^{f(id_{new})\cdot p}$. After that, $\mathrm{P}_{id_{new}}$ can join in the next aggregation round.

Efficiency Improvement: To further improve the communication and computational efficiency of the g-SA algorithm, we adopt the single instruction multiple data (SIMD) technique to pack multiple messages together for encryption and aggregation. Specifically, since the plaintext space is far greater than the message space, it can pack one participant's multiple messages of different dimensions through the super-increasing sequence. Then, packed messages could be encrypted and aggregated as usual, and the single aggregation results could be extracted from corresponding slots.

When our g-SA algorithm is applied to FL, the aggregation server AS can only obtain the grouped aggregation models but not individual local models, which can facilitate poisoning detection while guaranteeing the privacy of participants. Second, it is well adapted to the need of dynamic grouping in our framework. When the grouping rule changes, all participants can still use the same suite of keys, and the cryptosystem does not need to be re-initialized. Moreover, every participant in g-SA is not associated with others, i.e., the calculation complexity of message encryption is independent of the number of participants, which can keep efficient when facing large-scale participants.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

Remark: Note that although g-SA is a threshold algorithm, it has some differences from threshold homomorphic encryption [26]. In threshold homomorphic encryption, all messages are encrypted by the same public key, and any supported homomorphic operation can be executed over these ciphertexts. It divides the secret key into multiple parties, and the ciphertext can only be retrieved with partial decryption of more than S parties. Different from it, the g-SA algorithm generates multiple encryption keys by splitting a public parameter, where each participant encrypts its message with a different encryption key, and only the summation can trigger the trapdoor and be decrypted. For secure aggregation, both threshold homomorphic encryption and our g-SA algorithm can resist the collusion attack, but threshold homomorphic encryption cannot ensure honest aggregation execution, and its overhead is also higher.

Correctness: In the g-SA algorithm, since the key assignment of new participants is the same as the original participants, the correctness of participant joining is obvious, and its proof is omitted. Here we prove the correctness of the basic g-SA algorithm, and we also prove that the g-SA algorithm can still run normally when participants drop out.

Theorem 1: If $|\mathcal{G}_n^{(r)}| \geq S$, the g-SA algorithm can correctly obtain the aggregations over grouped participants, i.e., $A_n^{(r)} = \sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} W_{id}^{(r)}$ for $n = 1, \ldots, N$.

Proof: With equations $v = g^u \bmod \mathcal{N}^2$ and $sk_{id} = g'^{f(id)\cdot p} \bmod \mathcal{N}^2$, the ciphertext aggregation $[\![A_n^{(r)}]\!]$ of the group $\mathcal{G}_n^{(r)} (n = 1, \ldots, N)$ can be presented as

$$\begin{aligned}[\![A_n^{(r)}]\!] &= \prod\nolimits_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} [\![W_{id}^{(r)}]\!] \bmod \mathcal{N}^2 \\ &= \prod\nolimits_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} g^{W_{id}^{(r)}} v^{\gamma} {sk_{id}}^{\mu_{id}} \bmod \mathcal{N}^2 \\ &= g^{\sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} W_{id}^{(r)} + u\cdot\gamma'} g'^{\sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} p\cdot f(id)\cdot\mu_{id}} \bmod \mathcal{N}^2,\end{aligned}$$

where γ' is used to denote the summation of all random numbers. Since $|\mathcal{G}_n^{(r)}| \geq S$, according to the threshold secret sharing introduced in Section III-C, we can obtain

$$\begin{aligned}\sum\nolimits_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} f(id) \cdot \mu_{id} &= \sum\nolimits_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} k \cdot f(id) \prod_{\mathrm{P}_{id'} \in \mathcal{G}_n^{(r)}}^{id' \neq id} \frac{-id'}{id - id'} \\ &= 0 \bmod q\end{aligned}$$

and $[\![A_n^{(r)}]\!] = g^{\sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} W_{id}^{(r)} + u\cdot\gamma'} g'^{k'\cdot N} \bmod \mathcal{N}^2$, where k and k' are integers.

Then, $[\![A_n^{(r)}]\!]$ is in the form of a Paillier ciphertext. After decryption with msk, we have $A_n^{(r)} = \sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} W_{id}^{(r)} + u \cdot \gamma' \bmod u$. Since $W_{id}^{(r)} \ll u$, we finally obtain $A_n^{(r)} = \sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} W_{id}^{(r)}$, and the proof holds. □

Theorem 2: If $|\mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}| > S$, the messages of the remaining participants in $\mathcal{G}_n^{(r)}$ can be aggregated even though multiple participants in $\mathcal{L}_{drop}^{(r)}$ drops out during the r-th aggregation, i.e., $A_n^{(r)} = \sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}} W_{id}^{(r)}$.

Proof: With the recovery keys, the ciphertext aggregation $[\![A_n^{(r)}]\!]$ is

$$\begin{aligned}[\![A_n^{(r)}]\!] &= \prod_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}} [\![W_{id}^{(r)}]\!] \cdot \prod_{\mathrm{P}_{id} \in \mathcal{L}_{drop}^{(r)}} rk_{id} \bmod \mathcal{N}^2 \\ &= g^{\sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}} W_{id}^{(r)} + u\cdot\gamma'} g'^{k'N} \bmod \mathcal{N}^2\end{aligned}$$

It can be decrypted as the same in Theorem 1, and we obtain $A_n^{(r)} = \sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}} W_{id}^{(r)}$. □

B. Poisoning Defense Mechanism

In order to defend against poisoning attacks, we first design a poisoning detection method to find groups with malicious participants. Then, by combining the proposed random grouping method and reputation system, we can phase out malicious participants joining training and obtain a high-quality global model.

1) Random Grouping: In ELFL, participants are randomly and evenly grouped for every training round, and every participant can only exist in one group. After grouping, there are groups with malicious participants and groups without malicious participants.

Since our poisoning detection method is directly oriented to groups, the random grouping can avoid the condition that benign participants are always in the same group with malicious ones so that their local models are always removed from the global model and unable to make a contribution. Moreover, the random grouping can accurately find and phase out malicious participants. For multiple training rounds, if the group containing a certain participant is always detected as malicious, we can determine with a high probability that it is a malicious participant.

To achieve it, the number of groups N should be set suitably to balance security and privacy. If N is bigger, the number of participants in one group will decrease, and privacy preservation will be weakened. On the contrary, if N is smaller, there will be a larger proportion of malicious groups, resulting in a majority of malicious groups and influencing security. Therefore, KC first determines a minimum number of participants in each group, written as S, and the grouping needs to satisfy the conditions $N > 2 \cdot T$, $(1 - \rho) \cdot K \geq T$, $|\mathcal{L}^{(r)}| = N \cdot L$, and $L \geq S$.

2) Poisoning Detection: In our framework, AS uses its validation data $\mathcal{D}_v$ to evaluate the performance of all grouped aggregations and distinguish the poisonous and benign aggregations.

There are many works that directly evaluate the prediction accuracy of models on validation data, and the models causing an accuracy drop are identified as poisonous ones. However, in ELFL, since even malicious groups also contain multiple benign local models, the accuracy drop caused by the poisonous local models on the aggregations may not be significant.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

Therefore, in order to amplify the effect of poisonous local models, we adopt the output of the model outputs as the criterion distinguishing poisonous models. Specifically, AS propagates forward its validation samples on every grouped aggregation model $A_n^{(r)}$ and obtains the output, written as $o_n^{(r)}$. Then, AS calculates the cosine similarity between all pairs in $\{o_n^{(r)}\}_{n=1}^N$ and finds the outliers, which means the corresponding groups contain malicious participants.

3) Reputation System: Since the poisoning detection is not completely accurate, coupled with the fact that our framework can only detect the possible poisonous aggregations containing malicious participants, we cannot fully determine whether a certain participant is malicious. Therefore, we design a probabilistic reputation system to phase out malicious participants.

In the phase of parameter initialization, KC first sets a participant reputation list $\mathcal{R} = \{\Delta_{id1}, \Delta_{id2}, \ldots\}$ and the reputation score Δ_{id} of each participant is initialized to 0. Then, Δ_{id} is updated in every training round, and that of participants in the malicious group list $\mathcal{M}^{(r)}$ will decrease. Thus, a larger reputation score means that one participant is more likely to be benign, which can be considered as the criterion for selecting participants.

Through just a few training rounds in FL, as long as the accuracy of poisoning detection reaches a certain height, the reputation of malicious participants will decrease, and they will be excluded from training candidates. Meanwhile, benefiting from the random grouping method, benign participants will not always be in a group with malicious participants, so their reputation scores will not be too low to participate in training. Therefore, by combining the random grouping method and reputation system, ELFL can gradually prevent malicious participants from joining training and weaken poisonous models' damage to the global model.

Remark: As ELFL supports participants joining and quitting, when a malicious participant gets a low reputation score, and it is almost impossible to join the training, it may quit and rejoin our framework with a new identifier and reputation score. In order to avoid this situation, the initial reputation score of new participants is set as the median value of current reputation scores. In this way, a new malicious participant can be quickly excluded, and it will only influence a few training rounds.

VI. Analysis

This section analyzes the security and complexity of ELFL.

A. Security Analysis

We first prove the semantic security of the g-SA algorithm. Then, we give the definition of selective security and prove that ELFL could achieve the selective security in two cases with corrupted and dropped participants.

1) Semantic Security of the g-SA Algorithm: We prove the semantic security of the g-SA algorithm in a game-based paradigm. Specifically, we assume that there is a probabilistic probabilistic polynomial time (PPT) adversary $\mathcal{A}$ and a challenger $\mathcal{C}$, which execute the following interactions.

1) The challenger $\mathcal{C}$ executes the key generation function to obtain the public key $mpk = (g, g', v, \mathcal{N})$, aggregation key $msk = (\lambda, u)$, and a secret key sk_{id}. Then, mpk and msk are sent to the adversary $\mathcal{A}$.
2) $\mathcal{A}$ selects $m_0, m_1 \in \{0,1\}^{\frac{\kappa}{2}}$, which are sent to $\mathcal{C}$.
3) After receiving m_0 and m_1, $\mathcal{C}$ randomly selects $b \in \{0,1\}$ and calculates $[\![m_b]\!] = g^{m_b} v^{\gamma} (sk_{id})^{\mu_{id}}$, where γ and μ_{id} are randomly selected from $\{0,1\}^{\frac{\kappa}{2}-1}$ and $\mathbb{Z}_q$. $\mathcal{C}$ sends $[\![m_b]\!]$ to $\mathcal{A}$.
4) $\mathcal{A}$ guesses $b = 0$ or $b = 1$ from $[\![m_b]\!]$ and sends the result $b' \in \{0,1\}$ to $\mathcal{C}$, where the probability of a successful guess is written as $\Pr(b' = b) = \frac{1}{2} + \psi$.

Definition 1. (Semantic Security): If ψ is a negligible probability in $\Pr(b' = b)$, the g-SA algorithm is semantic security against the chosen plaintext attack (CPA).

Proof: From the perspective of $\mathcal{A}$, it can obtain the public key $mpk = (g, g', v, \mathcal{N})$, the aggregation key $msk = (\lambda, u)$, and a ciphertext $[\![m_b]\!]$. However, since $[\![m_b]\!] = g^{m_b} v^{\gamma} (sk_{id})^{\mu_{id}}$ is not a valid ciphertext of the Paillier encryption, which cannot trigger the trapdoor, $\mathcal{A}$ cannot use λ to decrypt it. Thus, to distinguish $[\![m_b]\!]$, $\mathcal{A}$ needs to eliminate the random number γ in $[\![m_b]\!]$, i.e., find an integer θ satisfying $g^{u\theta} = 1 \bmod \mathcal{N}^2$. Meanwhile, the plaintext m_b cannot be affected by θ, i.e., it requires θ to satisfy $g^{\theta} \neq 1 \bmod \mathcal{N}^2$. However, based on Definition 2, there is no such a θ that simultaneously satisfies the two conditions, so ψ is negligible, and the g-SA algorithm is semantic security. □

Definition 2: In the g-SA algorithm, there is no integer θ that simultaneously satisfies $g^{u\theta} = 1 \bmod \mathcal{N}^2$ and $g^{\theta} \neq 1 \bmod \mathcal{N}^2$.

Proof: $g^{u\theta} = 1 \bmod \mathcal{N}^2$ means $u\theta = k\phi(\mathcal{N})\mathcal{N}$, where k is an integer. Then, according to $gcd(u, \phi(\mathcal{N})) = 1$ and $\mathcal{N} = pq$, u is not a factor of $\phi(\mathcal{N})\mathcal{N}$, so we can obtain $\theta = k'\phi(\mathcal{N})\mathcal{N}$ and $k'u = k$, where k' is also an integer. However, $\theta = k'\phi(\mathcal{N})\mathcal{N}$ means $g^{\theta} = 1 \bmod \mathcal{N}^2$, which is contradictory to $g^{\theta} \neq 1 \bmod \mathcal{N}^2$. Therefore, the proof holds. □

Definition 3. (Selective Security): In the simulation paradigm, there is a PPT adversary $\mathcal{A}$ that controls some parties and attacks the real and ideal execution of our scheme, respectively. In ideal execution, a simulator $\mathcal{S}$ corrupts the same parties as $\mathcal{A}$ and simulates the ideal process. Our scheme is selective security *iff* the probability that $\mathcal{A}$ could distinguish the real and ideal executions is negligible.

2) (Case 1) Selective Security With Corrupted Participants: According to the security model defined in Section II-B, we assume that $\mathcal{A}$ corrupts AS and β participants in $\mathcal{L}_{corr}^{(r)}$ ($\beta < T - S$), and we prove the security in the worst case, i.e., corrupted participants are divided into the same group $\mathcal{G}_{n'}^{(r)}$. Then, we can define the leakage function $L(\mathcal{S})$ from the perspective of $\mathcal{S}$.

Leakage function: The information leaked to $\mathcal{S}$ is $L(\mathcal{S}) = \{\mathcal{L}^{(r)}, mpk, msk, \{sk_{id}, W_{id}^{(r)}\}_{\mathrm{P}_{id} \in \mathcal{L}_{corr}^{(r)}}, \{A_n^{(r)}\}_{n=1}^N\}$.

Ideal world: In the ideal world, the simulator $\mathcal{S}$ interacts with AS and participants in $\mathcal{L}^{(r)}$ as follows.

- *Parameter initialization:* $\mathcal{S}$ publishes the public key mpk and sends the secret key msk to AS.
- *Participant grouping:* $\mathcal{S}$ announces $\mathcal{L}^{(r)} = \{\mathcal{G}_1^{(r)}, \ldots, \mathcal{G}_N^{(r)}\}$.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

- *Local model uploading:* Upon receiving $[\![W_{id}^{(r)}]\!]$ from $\mathrm{P}_{id} \in \mathcal{L}^{(r)}$, for each group $\mathcal{G}_n^{(r)}(n = 1, \ldots, N)$, $\mathcal{S}$ first generates an aggregated ciphertext $[\![A_n'^{(r)}]\!] = g^{A_n^{(r)}} v^{\gamma} g'^{N} \bmod \mathcal{N}^2$. Then, $\mathcal{S}$ selects $L-1$ random integers $\{[\![W_{id2}'^{(r)}]\!], \ldots, [\![W_{idL}'^{(r)}]\!]\}$ for the last $L-1$ participants in $\mathcal{G}_n^{(r)}$, which have the same length as $[\![W_{id}^{(r)}]\!]$. After that, $\mathcal{S}$ calculates $[\![W_{id1}'^{(r)}]\!] = [\![A_n'^{(r)}]\!] \cdot (\prod_{i=2}^{S} [\![W_{idi}'^{(r)}]\!])^{-1}$. Finally, $\mathcal{S}$ sends $\{[\![W_{id}'^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)}}$ to AS on behalf of participants.
- *Grouped aggregating:* For each group $\mathcal{G}_n^{(r)} \in \mathcal{L}^{(r)}$, AS computes $[\![A_n'^{(r)}]\!] = \prod_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} [\![W_{id}'^{(r)}]\!] \bmod \mathcal{N}^2$. Then, AS decrypts $[\![A_n'^{(r)}]\!]$ with msk and obtains $A_n'^{(r)}$.

The view in real world $View_{\mathcal{A}}^{Real}$ contains $\{\{A_n^{(r)}\}_{n=1}^{N}\}, \{[\![W_{id}^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{corr}^{(r)}}\}$. In the ideal world, the view of $\mathcal{A}$ is $View_{\mathcal{A}}^{Ideal} = \{\{A_n'^{(r)}\}_{n=1}^{N}\}, \{[\![W_{id}'^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{corr}^{(r)}}\}$. On the basis of the two views, we formally prove our selective security.

Theorem 3: When AS colludes with $\beta(\beta < T - S)$ participants, the ELFL framework is selectively secure with the leakage $L(\mathcal{S})$.

Proof: In order to prove the selective security, we prove that every component in the real and ideal views are indistinguishable.

- $\mathcal{A}$ cannot distinguish $\{A_n'^{(r)}\}_{n=1}^{N}$ from $\{A_n^{(r)}\}_{n=1}^{N}$. First we can reduce $[\![A_n'^{(r)}]\!] = \prod_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}} [\![W_{id}'^{(r)}]\!] = g^{A_n^{(r)}} v^{\gamma} g'^{N} \bmod \mathcal{N}^2$. Then, according to Theorem 1, $\mathcal{A}$ can obtain $A_n'^{(r)} = A_n^{(r)}$ with msk. Thus, $A_n^{(r)}$ and $A_n'^{(r)}$ have the same value, and $\mathcal{A}$ cannot distinguish them.
- $\mathcal{A}$ cannot distinguish $\{[\![W_{id}'^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{corr}^{(r)}}$ from $\{[\![W_{id}^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{corr}^{(r)}}$. Based the semantic security of the g-SA algorithm proved in Definition 1, $\mathcal{A}$ cannot distinguish $[\![W'^{(r)}_{id}]\!]$ from $[\![W_{id}^{(r)}]\!]$.

Therefore, with leakage $\mathcal{L}(S)$, the ideal view of ELFL is indistinguishable from the real view, and ELFL achieves the selective security when AS and β participants are collusive. □

3) (Case 2) Selective Security With Dropped Participants: In this section, we assume that participants in $\mathcal{L}_{drop}^{(r)}$ drop out during the r-th round. Then, we also define the leakage function $L(\mathcal{S})$.

Leakage function: $L(\mathcal{S}) = \{\mathcal{L}^{(r)}, mpk, msk, \{rk_{id}\}_{\mathrm{P}_{id} \in \mathcal{L}_{drop}^{(r)}}, \{A_n^{(r)}\}_{n=1}^{N}\}$.

Ideal world. The interactions are as follows.

- *Parameter initialization:* $\mathcal{S}$ publishes the public key mpk and sends the secret key msk to AS.
- *Participant grouping:* $\mathcal{S}$ announces $\mathcal{L}^{(r)} = \{\mathcal{G}_1^{(r)}, \ldots, \mathcal{G}_N^{(r)}\}$.
- *Local model uploading:* After receiving $\{[\![W_{id}^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{drop}^{(r)}}$, $\mathcal{S}$ calculates a ciphertext $[\![A_n'^{(r)}]\!] = g^{A_n^{(r)}} v^{\gamma} g'^{N} \bmod \mathcal{N}^2$ for each group $\mathcal{G}_n^{(r)}(n = 1, \ldots, N)$. Then, $\mathcal{S}$ generates a random number $[\![W_{id}'^{(r)}]\!]$ with the same length as $[\![W_{id}^{(r)}]\!]$ for each participant $\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}$, and these numbers satisfy $[\![A_n'^{(r)}]\!] = \sum_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}} [\![W_{id}'^{(r)}]\!]$. After that, $\mathcal{S}$ sends $\{[\![W_{id}'^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{drop}^{(r)}}$ and $\{rk_{id}\}_{\mathrm{P}_{id} \in \mathcal{L}_{drop}^{(r)}}$ to AS.
- *Grouped aggregation:* For every group $\mathcal{G}_n^{(r)}$, AS computes $[\![A_n'^{(r)}]\!] = \prod_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)} \setminus \mathcal{L}_{drop}^{(r)}} [\![W_{id}'^{(r)}]\!] \cdot \prod_{\mathrm{P}_{id} \in \mathcal{G}_n^{(r)}, \mathcal{L}_{drop}^{(r)}} rk_{id} \bmod \mathcal{N}^2$ and decrypts it to $A_n^{(r)}$.

Therefore, with dropped participants, we have $View_{\mathcal{A}}^{Real} = \{\{A_n^{(r)}\}_{n=1}^{N}\}, \{[\![W_{id}^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{drop}^{(r)}}\}$ and $View_{\mathcal{A}}^{Ideal} = \{\{A_n'^{(r)}\}_{n=1}^{N}\}, \{[\![W_{id}'^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{drop}^{(r)}}\}$.

Theorem 4: The ELFL framework is selectively secure with the leakage $L(\mathcal{S})$, even though participants in $\mathcal{L}_{drop}^{(r)}$ drop out.

Proof: We also prove the indistinguishability between the real and ideal views.

- $\mathcal{A}$ cannot distinguish $\{A_n'^{(r)}\}_{n=1}^{N}$ from $\{A_n^{(r)}\}_{n=1}^{N}$. According to Theorem 2, we have $[\![A_n'^{(r)}]\!] = g^{A_n^{(r)} + u \cdot \gamma} g'^{kN} \bmod \mathcal{N}^2$. Therefore, through decryption, we can obtain $A_n'^{(r)} = A_n^{(r)}$, which can not be distinguished.
- $\mathcal{A}$ cannot distinguish $\{[\![W_{id}'^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{drop}^{(r)}}$ from $\{[\![W_{id}^{(r)}]\!]\}_{\mathrm{P}_{id} \in \mathcal{L}^{(r)} \setminus \mathcal{L}_{drop}^{(r)}}$. As the proof of Definition 1, $\mathcal{A}$ cannot break the indistinguishability of the g-SA algorithm, and it cannot distinguish $[\![W_{id}'^{(r)}]\!]$ and $[\![W_{id}^{(r)}]\!]$.

Therefore, the proof holds. □

Since the recovery keys $\{rk_{id}\}_{\mathrm{P}_{id} \in \mathcal{L}_{drop}^{(r)}} \in L(\mathcal{S})$ are generated for dropped participants, and ELFL uses the same suite of keys in every training round, the adversary may try to recover the past local models of dropped participants through their recovery keys. However, the dropping handling method of ELFL will generate a random number γ to perturb rk_{id}, so the adversary cannot find any relationship between the secret key and recovery key and cannot recover the local models.

Moreover, since grouped aggregations $\{A_n^{(r)}\}_{n=1}^{N} \in L(\mathcal{S})$ are the summation of local models for not dropped participants, we need to ensure that the number of dropped participants in each group is less than $L - S$, otherwise KC will not provide recovery keys for them.

B. Complexity Analysis

We analyze the numerical complexity of ELFL in theory and compare it with SAFELearning and SBRDL. Note that we only count the computations over ciphertexts or secret shares as they cost more, and that over plaintexts, such as local model training and poisoning detection over aggregations, are omitted.

- *Computational cost of participants:* Each participant P_{id} in ELFL needs to encrypt M model parameters, and every encryption is independent of the number of participants. Therefore, its computational complexity is $\mathcal{O}(M)$.
- *Communication overhead of participants:* During each training round, P_{id} receives the global model and uploads

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPUTATIONAL AND COMMUNICATION COMPLEXITY COMPARISON

Schemes	Participant		Aggregation Server	
	Computation	Communication	Computation	Communication
SAFELearning [18]	$\mathcal{O}(K+L^2+M)$	$\mathcal{O}(K+M)$	$\mathcal{O}(KM+KL)$	$\mathcal{O}(K^2+KM)$
SBRDL [19]	$\mathcal{O}(K+L^2+M)$	$\mathcal{O}(L+M)$	$\mathcal{O}(KM+KL)$	$\mathcal{O}(KL+KM)$
ELFL	$\mathcal{O}(M)$	$\mathcal{O}(M)$	$\mathcal{O}(KM)$	$\mathcal{O}(KM)$

the local model, both of which have M parameters, so the communication overhead is $\mathcal{O}(M)$.

- *Computational cost of AS:* AS computes the aggregations of M parameters, and each aggregation needs to compute the multiplications of $\rho \cdot K$ values. Therefore, its computational complexity is $\mathcal{O}(KM)$.
- *Communication overhead of AS:* During each round, AS receives and distributes models with M parameters to $\rho \cdot K$ participants, so its communication overhead is $\mathcal{O}(KM)$.

We list the numerical complexity of ELFL, SAFELearning, and SBRDL in Table II. We can find that both communication complexity and computational complexity of ELFL are lower than that of the other two schemes, which means that ELFL is more suitable for large-scale scenes.

VII. PERFORMANCE EVALUATION

In this section, we first introduce the experimental environment and setup. Then, we evaluate the performance of ELFL with different kinds of poisoning attacks and different malicious ratios. Finally, the computational and communication overhead is compared with existing representative schemes.

A. Experimental Setup

1) Implementation: Both the client-side and server-side programs of ELFL are implemented with Python 3.7.4. Specifically, the local training algorithm is based on the TensorFlow platform (version 2.5.0). The g-SA algorithm is implemented on top of the PHE library (version 1.4.0), and the outlier detection step adopts the clustering module in the Sklearn library (version 1.18.5).

2) Testbed: The programs of participants execute on Thinkpad P53 machines with Intel Core i5-9400H 2.5 GHz processors and 24 GB RAM, running Windows 11. The program of AS is conducted on a Dell Precision 7920 workstation equipped with Intel Xeon Gold 6226R 2.90 GHz processors and 256 GB RAM, running Ubuntu 20.04. Participants connect to AS through a WAN network with 100 Mbps bandwidth and 42 ms latency.

3) Datasets: Two real-world datasets are used to evaluate the performance of ELFL, namely handwritten digits recognition (MNIST) and image recognition (CIFAR-10). The MNIST dataset [27] contains 60000 grey-scale handwritten digit images from 0 to 9, where 50000 samples are used for training, and others are testing samples. Each sample in MNIST contains 28×28 pixels and one label indicating its true classification. The CIFAR-10 dataset [28] consists of 60000 color images in ten classes (airplane, cat, etc.), which is evenly divided into five training sets and one testing set. Each sample in CIFAR-10 contains 32×32 pixels and one label, and each pixel contains three channels (RGB).

4) Model Architectures and Hyperparameters: The model for the MNIST dataset aims to predict which digit the sample is, which consists of two fully connected layers with 784×100 and 100×10 parameters, respectively. The LeNet 5 model [29] is used for the CIFAR-10 dataset to predict the true classification of samples, which contains two convolution layers and three fully-connected layers. In the effectiveness experiments, the number of active participants is set as 100 ($K = 100$), and the training samples are evenly and randomly divided into them. Meanwhile, we randomly select 100 data samples as the validation samples for poisoning detection. According to the requirements of the random grouping method, we set the number of malicious participants varying from 5 to 25, the number of groups varying from 11 to 51, and the ratio of selected participants in every round varying from 90% to 70%. Moreover, we set the learning rate $\alpha = 0.05$, precision parameter $\epsilon = 1000$, local epoch $E = 3$ for the MNIST dataset and $E = 1$ for the CIFAR-10 dataset. For security, we set the security parameter $\kappa = 1024$, so the plaintext space is 512 bits, and each ciphertext could pack 15 values.

5) Attack Implementation: We adopt two representative data poisoning attacks to evaluate the performance of ELFL, which are the label-flipping attack and the backdoor attack. The label-flipping attack attempts to mislead the samples with a source class to a target class, which is simulated in our experiments by flipping the labels of poisonous samples from 1 to 9. The backdoor attack aims to embed a trigger in the global model. In our experiments, the malicious participants randomly select 80% of training samples, change their 5×5 pixels at the lower right corner to the maximum values as the trigger, and set their labels to 9 as the attack target. Moreover, for implementing successful attacks, the learning rate of malicious participants is set to 10 times that of benign ones.

6) Metrics: Here we introduce the performance metrics used to evaluate the effectiveness and efficiency of ELFL.

- *Prediction Accuracy (Acc)/Loss:* The prediction accuracy is the ratio of the number of successfully predicted samples to the total number of testing samples, and the prediction loss is measured by Mean Absolute Error (MAE).
- *Attack Success Ratio (ASR):* The attack success ratio is the ratio of the number of samples successfully attacked to that of the total attack samples. For the label-flipping attack, the attack samples belong to the source class, and a successful attack means that it is misclassified to the target class. For the backdoor attack, the attack sample has a trigger, which attacks successfully if it is classified to the target class.
- *Malicious Ratio per Round (MRpR):* Since the reputation system will phase out malicious participants, MRpR is a

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

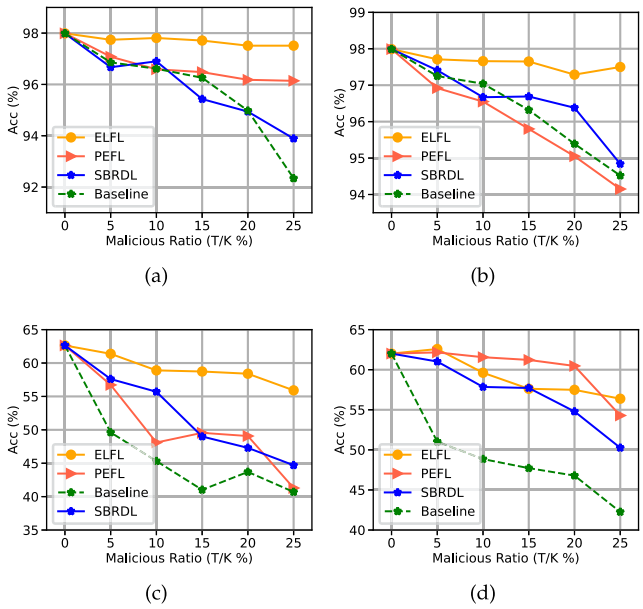

Fig. 4. Prediction accuracy comparison with different ratios of malicious participants. (a) Label-flipping attack (MNIST); (b) Backdoor attack (MNIST); (c) Label-flipping attack (CIFAR-10); (d) Backdoor attack (CIFAR-10).

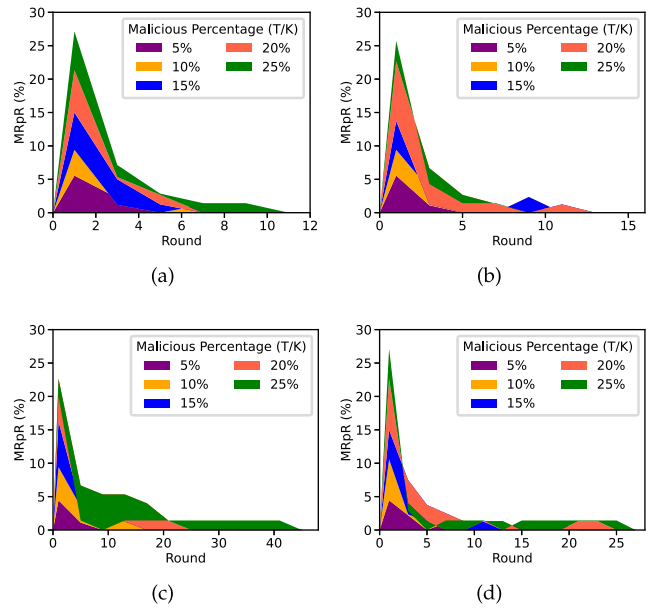

Fig. 5. Ratio of malicious participants during each training round in ELFL. (a) Label-flipping attack (MNIST); (b) Backdoor attack (MNIST); (c) Label-flipping attack (CIFAR-10); (d) Backdoor attack (CIFAR-10).

metric in ELFL that denotes the ratio of the number of malicious participants to that of total training participants.

- *Detection Success Ratio (DSR):* The DSR is the averaging ratio of the number of successfully detected groups (containing benign and malicious groups) to total groups.
- *Computational/communication overhead:* The computational cost is measured by the running time of programs, and the communication overhead is the size of transmitted messages.

7) Baseline Scheme: To evaluate the effectiveness of ELFL, we compare its prediction accuracy with the vanilla FL scheme (Baseline) [1], a notable HE-based FL scheme (PEFL) [12], and a masking-based FL scheme with grouped aggregation (SBRDL) [19]. In the vanilla FL scheme, there is no method adopted for privacy preservation or poisoning resistance. Moreover, we also evaluate and compare our computational and communication efficiency with the vanilla double masking scheme (DM) [8], PEFL, and another masking-based grouped FL scheme SAFELearning [18]. Since SAFELearning detects poisoning attacks in the training rounds close to convergence, which is a different assumption from ours, we only compare the efficiency but not accuracy with it.

B. Effectiveness Evaluation

This section evaluates the effectiveness of ELFL against data poisoning attacks.

1) Prediction Accuracy Varying Different Malicious Ratios: Fig. 4 compares the prediction accuracy of baseline, PEFL, SBRDL, and ELFL under different ratios of malicious participants. First, we observe that the accuracy of ELFL just has a little drop when the malicious participants increase, but that of the other three schemes decreases significantly, which is because ELFL could detect and phase out malicious participants only through a few training rounds, and they hardly influence the global model. Second, the accuracy drop caused by the backdoor attack is lower than that caused by the label-flipping attack. We analyze this because the wrong classification by the backdoor attack needs the trigger of the modified pixel values, but the testing data do not contain triggers. Meanwhile, for the label-flipping attack, unmodified data with source labels would be misled to target classifications. Moreover, the accuracy on the CIFAR-10 decreases more quickly than that on the MNIST dataset with malicious ratios growing, which means that more complex models are less resistant to poisoning attacks. Despite this, the prediction accuracy of ELFL is markedly higher than others for both two kinds of poisoning attacks and datasets, which demonstrates its strong poisoning-resistant ability.

2) Malicious Ratio in Each Training Round: Fig. 5 plots the ratio of malicious participants during each training round. Obviously, when the percentage of malicious participants in all active participants is larger, the malicious ratio in the initial training round is larger. However, no matter if there are many or few malicious participants, ELFL can quickly lower their reputations based on the poisoning detection method and reputation system and exclude them from the training candidates. Through only less than 15 training rounds for the MNIST dataset (45 for the CIFAR-10 dataset), there is no malicious participant that is selected for model training, and then the global model could be constructed by all benign participants, which is the reason that ELFL is more accurate than other schemes.

3) Defensive Performance: Fig. 6 shows the ASR and DSR under different malicious ratios. We can first find that even though AS only has 100 validation samples, our poisoning detection method could detect the malicious groups with an accuracy of over 90%, and it hardly decreases with the malicious

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

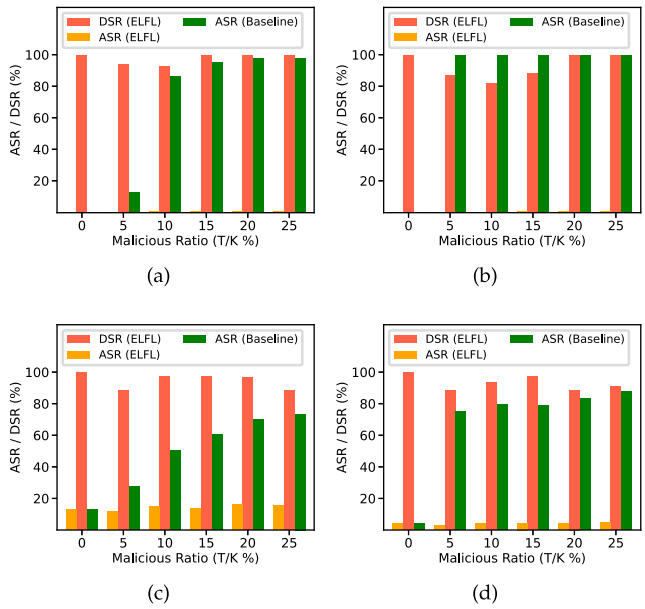

Fig. 6. Ratio of successful attack and detection. (a) Label-flipping attack (MNIST); (b) Backdoor attack (MNIST); (c) Label-flipping attack (CIFAR-10); (d) Backdoor attack (CIFAR-10).

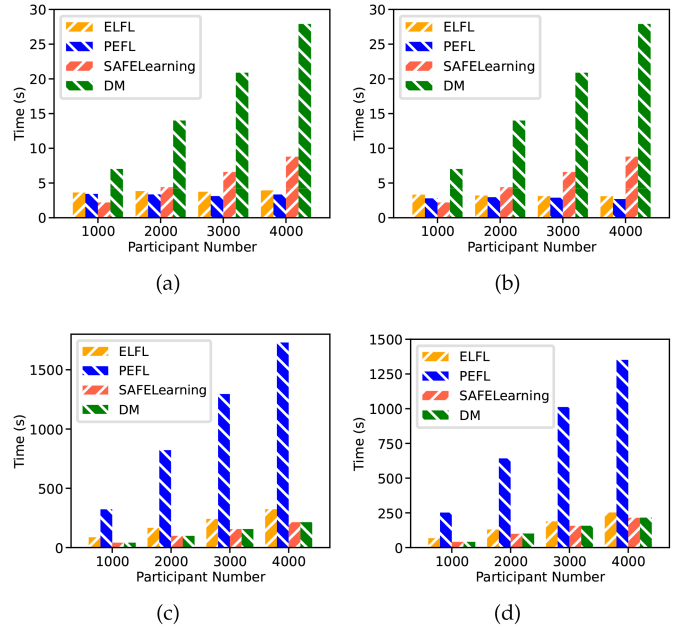

Fig. 7. Computational cost of one training round varying K. (a) Computational cost of the participant (MNIST); (b) Computational cost of the participant (CIFAR-10); (c) Computational cost of AS (MNIST); (d) Computational cost of AS (CIFAR-10).

ratio increasing. Then, in the vanilla FL scheme, the ASR of the label-flipping attack increases when the malicious ratio increases, and that of the backdoor attack is always high, even though the ratio of malicious participants is low. Despite this, for ELFL on the MNIST dataset, the ASR of both two kinds of attacks is almost 0. For the CIFAR-10 dataset, though some attacks are successful, the ASR of ELFL is the same as that with no malicious participants, which is because the prediction accuracy for CIFAR-10 is relatively low, and it originally has a certain probability of misclassifying the testing data into the attack targets. Therefore, the experimental results demonstrate that the model in ELFL is hardly influenced by malicious participants, and it could suppress the attack success ratio like there is no attack.

C. Efficiency Evaluation

1) Computational Cost: Fig. 7 shows the computational cost in one training round varying K. From Fig. 7(a) and (b), we can find that as the number of participants increases, the computational cost of each participant is almost changeless in ELFL and PEFL, but that of the other two masking-based schemes increases linearly. This result confirms our theoretical analysis, and it demonstrates that ELFL is more suitable for the scene with plenty of participants. Meanwhile, the computational cost of ELFL and PEFL is quite close, since both of them protect local models based on the Paillier encryption. Moreover, we find that the computational cost of two masking-based schemes hardly changes for two datasets with different model scales, but that of ELFL and PEFL increases with the model scale growing (the models of MNIST and CIFAR-10 datasets contain 79510 and 62006 parameters, respectively), which is because encryption operations spend more time than masking operations.

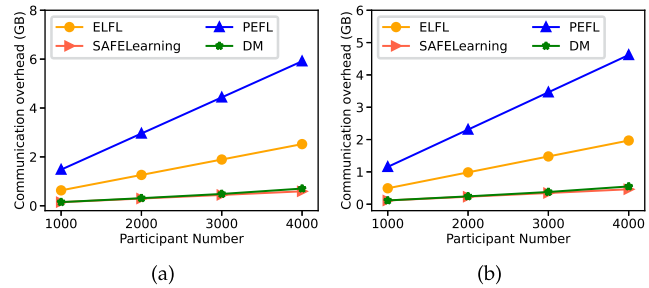

Fig. 8. Communication overhead of AS varying K. (a) MNIST dataset; (b) CIFAR-10 dataset.

Fig. 7(c) and (d) plot the computational cost of AS. We can find that the computational cost grows as the number of participants increases for all four schemes. Moreover, the computational cost of PEFL is markedly higher than ELFL since it adopts complex cipher-based protocols to detect poisoning attacks.

2) Communication Overhead: Since AS needs to send and receive models from all participants, its communication overhead becomes a main bottleneck, and Fig. 8 shows the communication overhead of AS varying K. We can find that for all four schemes, the communication overhead linearly increases as S grows. ELFL adopts the encryption technique for privacy preservation, whose ciphertext space is four times the plaintext space, but the masking technique does not expand the plaintext space, so the communication overhead of ELFL is also four times the other two masking-based schemes. However, the communication overhead of ELFL is markedly lower than the HE-based scheme PEFL because ELFL requires just one server, but PEFL needs frequent interaction between two servers.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

TABLE III
FUNCTIONALITY COMPARISON OF FL SCHEMES

Schemes	Privacy-preserving	Poisoning-resistant	Single server	Dynamic grouping	Complexity
VerifyNet [30]	✓	✗	✓	-	High
Krum [11]	✗	✓	✓	-	Low
PEFL [12]	✓	✓	✗	-	High
SBRDL [19]	✓	✓	✓	✗	High
SAFELearning [18]	✓	✓	✓	✗	High
ELFL	✓	✓	✓	✓	Low

In summary, ELFL has a powerful ability against poisoning attacks, and its efficiency can be compared favorably to representative masking-based schemes. Moreover, while supporting dynamic grouping, the complexity of ELFL is lower than that of masking-based schemes. Therefore, ELFL can be utilized in most real scenes, especially for large-scale participants whose devices do not have strong computing or communication capabilities.

VIII. Related Works

This section reviews some representative FL schemes that aim to defend against inference attacks, poisoning attacks, or both.

A. Privacy-Preserving FL Schemes

Recently, there have been many privacy-preserving FL schemes to ensure the confidentiality of local models via secure aggregation. According to different secure techniques, these schemes can be classified into differential privacy [31] or cryptography-based ones. Based on differential privacy, Abadi et al. [32] presented a DP-SGD algorithm to protect local models by adding Gaussian noises to them, and they provided a moment accounting method to accurately measure the privacy budget in FL. Cryptography-based schemes use some techniques like HE [9], [33], [34], masking [8], [30], or MPC [15], [35] to protect individual local models. For example, based on the double masking technique, Xu et al. [30] designed an SA protocol for FL, which could keep the confidentiality of individual local models and tolerate participants dropping out. Zhao et al. [36] used HE as the underlying secure technique and proposed a privacy-preserving decentralized FL framework, where the models are propagated and updated over ciphertexts. Cryptography-based schemes can obtain a lossless global model, but their operations may cause huge computational and communication overhead.

B. Poisoning-Resistant FL Schemes

Poisoning-resistant FL schemes try to weaken the effect of poisonous local models on the global model, which can be divided into two mainstream ideas. One class of schemes designs Byzantine-robust aggregation algorithms for weakening the impact of poisoning models [11], [37], [38]. For example, Blanchard et al. [11] proposed a Byzantine-resilient aggregation rule named Krum, which selects the central local model that is the most similar to others as the global model. Yin et al. [37] adopted mean, median, and trimmed mean to aggregate local models and evaluate their robustness on logic regression and convolutional neural network models. The other class of methods is to detect malicious models via model analysis or evaluation techniques and then eliminate detected ones from the global model [39], [40], which is based on the assumption that the poisonous models are statistically different from benign ones. For example, Tan et al. [41] proposed a verify-before-aggregation procedure, where the aggregation server evaluates the accuracy on a validation dataset after adding each local model, and removes local models that cause accuracy loss.

C. Privacy-Preserving and Poisoning-Resistant FL Schemes

Privacy-preserving and poisoning-resistant FL schemes aim to defend against poisoning attacks while protecting the privacy of local models. To achieve it, Liu et al. [12] proposed a privacy-enhancing FL scheme named PEFL, which protects the local models by HE and detects poisonous models over HE ciphertexts. Specifically, based on the collaboration of two non-collusive servers, they designed the cipher-based Pearson correlation coefficient and median calculation protocols, which could calculate the distance between encrypted local models and a benchmark model, and then they could distinguish the poisonous and benign local models according to the distance. Chen et al. [13] also adopted HE as the privacy-preserving technique and proposed a cipher-based collaborative multi-layer perception scheme, which could evaluate the quality of local models by calculating the model accuracy over ciphertext local models and a validation dataset. Note that their cipher-based calculations also need the cooperation of two servers, namely aggregation server and evaluator.

There are also some two-step or multi-step aggregation schemes similar to our scheme. For example, Velicheti et al. [19] designed a secure byzantine-robust scheme to aggregate local models in two steps. It also randomly groups participants in each training round and uses double masking and robust aggregation techniques like Krum to ensure privacy and security. Zhang et al. [18] proposed a multi-step SA scheme, named SAFELearning. Specifically, it first uses a tree structure to divide participants and initializes the masking keys for participants in every leaf node. During each training round, it securely aggregates the local models in each leaf via the double masking technique, and detects malicious aggregations by observing their higher bits. However, their computational complexity is as high as $\mathcal{O}(L^2)$, and they cannot support dynamic grouping.

Different from the above schemes, ELFL does not need time-consuming calculation of cipher-based similarity, and it requires neither multiple servers nor communication between participants. Meanwhile, by specially designing the g-SA algorithm, ELFL supports dynamic grouping well and continuously

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

uses a suite of keys throughout the training process. Moreover, participants in ELFL have a low computational and communication complexity. Table III provides a functionality comparison of FL schemes.

IX. Conclusion

In this paper, we have proposed ELFL to achieve efficient and privacy-preserving federated learning against poisoning adversaries. Based on the g-SA algorithm and poisoning defense mechanism, ELFL could protect the privacy of individual local models while removing the destructive effect of malicious participants on the global model. Detailed security analysis and experimental evaluation demonstrate that ELFL provides strong privacy against inference attacks, reliable security against data poisoning attacks, and high efficiency against large-scale participants. In the future, we will attempt to adjust the grouping strategy so that ELFL can resist a larger ratio of malicious participants in FL.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.

[3] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, "A survey on security and privacy of federated learning," *Future Gener. Comput. Syst.*, vol. 115, pp. 619–640, 2021.

[4] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 14 747–14 756.

[5] H. Yin, A. Mallya, A. Vahdat, J. M. Alvarez, J. Kautz, and P. Molchanov, "See through gradients: Image batch recovery via gradinversion," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 16 337–16 346.

[6] Z. Tian, L. Cui, J. Liang, and S. Yu, "A comprehensive survey on poisoning attacks and countermeasures in machine learning," *ACM Comput. Surv.*, vol. 55, no. 8, pp. 166:1–166:35, 2023.

[7] V. Shejwalkar, A. Houmansadr, P. Kairouz, and D. Ramage, "Back to the drawing board: A critical evaluation of poisoning attacks on production federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1354–1371.

[8] K. A. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.

[9] J. Zhao et al., "CORK: A privacy-preserving and lossless federated learning scheme for deep neural network," *Inf. Sci.*, vol. 603, pp. 190–209, 2022.

[10] Y. Chen, L. Su, and J. Xu, "Distributed statistical machine learning in adversarial settings: Byzantine gradient descent," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 2, pp. 44:1–44:25, 2017.

[11] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 119–129.

[12] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, Aug. 2021.

[13] Y. Chen, X. Zhang, Y. Xie, M. Miao, and X. Ma, "CECMLP: New cipher-based evaluating collaborative multi-layer perceptron scheme in federated learning," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.*, Springer, 2021, pp. 79–99.

[14] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1639–1654, Apr. 2022.

[15] Y. Khazbak, T. Tan, and G. Cao, "MLGuard: Mitigating poisoning attacks in privacy preserving distributed collaborative learning," in *Proc. 29th Int. Conf. Comput. Commun. Netw.*, 2020, pp. 1–9.

[16] T. D. Nguyen et al., "FLGUARD: Secure and private federated learning," 2021, *arXiv:2101.02281*.

[17] C. Dong et al., "Privacy-preserving and Byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 889–904, Mar./Apr. 2024.

[18] Z. Zhang, J. Li, S. Yu, and C. Makaya, "SAFELearning: Enable backdoor detectability in federated learning with secure aggregation," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3289–3304, May 2023.

[19] R. K. Velicheti, D. Xia, and S. Koyejo, "Secure Byzantine-robust distributed learning via clustering," in *Proc. NeurIPS Workshop New Front. Federated Learn. Privacy Fairness Robustness Personalization Data Ownership*, 2021, pp. 1–9.

[20] O. Goldreich, S. Micali, and A. Wigderson, "How to play ANY mental game or a completeness theorem for protocols with honest majority," in *Proc. Int. Conf. Providing Sound Found. Cryptography*, 2019, pp. 307–328.

[21] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proc. Int. Conf. Mach. Learn.*, 2012, pp. 1467–1474.

[22] L. Truong et al., "Systematic evaluation of backdoor data poisoning attacks on image classifiers," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2020, pp. 3422–3431.

[23] I. J. Goodfellow, Y. Bengio, and A. C. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[24] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 1999, pp. 223–238.

[25] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[26] S. Truex et al., "A hybrid approach to privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 1–11.

[27] Y. LeCun, "The MNIST database of handwritten digits," 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[28] A. Krizhevsky, V. Nair, and G. Hinton, "The CIFAR-10 dataset," 2014. [Online]. Available: http://www.cs.toronto.edu/kriz/cifar.html

[29] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[30] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, Jul. 2020.

[31] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3/4, pp. 211–407, 2014.

[32] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.

[33] X. Zhang, A. Fu, H. Wang, C. Zhou, and Z. Chen, "A privacy-preserving and verifiable federated learning scheme," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–6.

[34] F. Wang, H. Zhu, R. Lu, Y. Zheng, and H. Li, "Achieve efficient and privacy-preserving disease risk assessment over multi-outsourced vertical datasets," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1492–1504, May/Jun. 2022.

[35] S. Kadhe, N. Rajaraman, O. O. Koyluoglu, and K. Ramchandran, "FastSecAgg: Scalable secure aggregation for privacy-preserving federated learning," 2020, *arXiv: 2009.11248*.

[36] J. Zhao, H. Zhu, F. Wang, R. Lu, Z. Liu, and H. Li, "PVD-FL: A privacy-preserving and verifiable decentralized federated learning framework," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2059–2073, May 2022.

[37] D. Yin, Y. Chen, K. Ramchandran, and P. L. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5636–5645.

[38] M. S. Özdayi, M. Kantarcioglu, and Y. R. Gel, "Defending against backdoors in federated learning with robust learning rate," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 9268–9276.

[39] J. Kang, Z. Xiong, D. Niyato, Y. Zou, Y. Zhang, and M. Guizani, "Reliable federated learning for mobile networks," *IEEE Wireless Commun.*, vol. 27, no. 2, pp. 72–80, Apr. 2020.

[40] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, "FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2022, pp. 2545–2555.

[41] J. Tan, Y. Liang, N. C. Luong, and D. Niyato, "Toward smart security enhancement of federated learning networks," *IEEE Netw.*, vol. 35, no. 1, pp. 340–347, Jan./Feb. 2021.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.

Jiaqi Zhao received the BEng degree in information security from Xidian University, Xi'an, Shaanxi, China, in 2020. He is currently working toward the PhD degree in cyberspace security with Xidian University. His research has been concerned with privacy-preserving machine learning.

Hui Zhu (Senior Member, IEEE) received the BSc degree from Xidian University, Xi'an, Shaanxi, China, in 2003, the MSc degree from Wuhan University, Wuhan, Hubei, China, in 2005, and the PhD degree from Xidian University, in 2009. He was a research fellow with the School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore, in 2013. Since 2016, he has been a professor with the School of Cyber Engineering, Xidian University. His current research interests include applied cryptography, data security, and privacy.

Fengwei Wang received the BSc and PhD degrees from Xidian University, in 2016 and 2021. In 2019, he was a visiting scholar with the Faculty of Computer Science (FCS), University of New Brunswick (UNB), Canada. Since 2021, he has been the lecturer with the School of Cyber Engineering, Xidian University, China. His research interests include the areas of applied cryptography, Big Data security, and privacy protection.

Yandong Zheng received the MS degree from the Department of Computer Science, Beihang University, China, in 2017, and the PhD degree from the Department of Computer Science, University of New Brunswick, Canada, in 2022. Since 2022, she has been an associate professor with the School of Cyber Engineering, Xidian University. Her research interests include cloud computing security, Big Data privacy, and applied privacy.

Rongxing Lu (Fellow, IEEE) received the PhD degree from the Department of Electrical & Computer Engineering, University of Waterloo, Canada, in 2012. He is Mastercard IoT research chair with the University Research Scholar, a professor with the Faculty of Computer Science (FCS), University of New Brunswick (UNB), Canada. Before that, he worked as an assistant professor with the School of Electrical and Electronic Engineering, Nanyang Technological University (NTU), Singapore from 2013 to 2016. He worked as a postdoctoral fellow with the University of Waterloo from 2012 to 2013. He was awarded the most prestigious "Governor General's Gold Medal", when he received his PhD degree; and won the 8th IEEE Communications Society (ComSoc) Asia Pacific (AP) Outstanding Young Researcher Award, in 2013. His research interests include applied cryptography, privacy enhancing technologies, and IoT-Big Data security and privacy. He has published extensively in his areas of expertise, and was the recipient of 9 best (student) paper awards from some reputable journals and conferences. Currently, he serves as the chair of IEEE ComSoc CIS-TC (Communications and Information Security Technical Committee), and the founding co-chair of IEEE TEMS Blockchain and Distributed Ledgers Technologies Technical Committee (BDLT-TC). He is the Winner of 2016-17 Excellence in Teaching Award, FCS, UNB.

Hui Li (Member, IEEE) received the BSc degree from Fudan University, in 1990, the MSc and PhD degrees from Xidian University, China, in 1993 and 1998, respectively. Since 2005, he has been the professor with the School of Telecommunication Engineering, Xidian University. His research interests include the areas of cryptography, wireless network security, information theory, and network coding. He served as TPC co-chair of ISPEC 2009 and IAS 2009, general co-chair of E-Forensic 2010, ProvSec 2011 and ISC 2011, honorary chair of NSS 2014, ASIACCS 2016.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:04:41 UTC from IEEE Xplore. Restrictions apply.